About this PDF

Built only from: **Artificial Intelligence and Machine Learning (Neural Networks and Deep Learning)** — https://nptel.ac.in/courses/106105077

Licence: CC BY-SA 4.0. Accessed .

Covers 11 of 11 syllabus sub-topics of this unit.

Perceptron

The perceptron is a mathematical processing unit inspired by biological neurons.

Definition

The perceptron is a mathematical processing unit inspired by biological neurons in the human brain, introduced by Frank Rosenblatt in 1958 as the fundamental computational building block of neural architectures.

Explanation

It takes an input vector (x = (x_1, x_2, dots, x_d)), multiplies each input by a corresponding synaptic weight (w = (w_1, w_2, dots, w_d)), aggregates these weighted inputs with an added scalar bias (b), and applies a non-linear threshold activation function (f). In the classical perceptron, the activation function is the Heaviside step function: (f(z) = 1) if (z ge 0), and (f(z) = 0) (or (-1)) if (z < 0). For binary classification with targets (y in -1, +1), weights and bias are updated iteratively for each misclassified training example ((x_i, y_i)) according to: (w := w + eta , y_i , x_i), (b := b + eta , y_i), where (eta) denotes the learning rate hyperparameter. If training data is linearly separable, the perceptron learning algorithm is guaranteed to converge to a separating hyperplane in a finite number of update steps. However, single-layer perceptrons cannot learn non-linearly separable functions, such as the fundamental XOR logic gate.

Example

The XOR logic gate is an example of a non-linearly separable function that a single-layer perceptron cannot learn.

Algorithm / Steps

1. Compute net input (z = w^T x + b = sum_j=1^d (w_j x_j) + b).
2. Compute output (y = f(z)).
3. If (y neq y_target), update weights and bias:
   - (w := w + eta , y_target , x)
   - (b := b + eta , y_target).
4. Repeat until convergence or a maximum number of iterations is reached.

Key points

- The perceptron is a linear classifier with a threshold activation function.
- It converges to a separating hyperplane if the training data is linearly separable.
- Single-layer perceptrons cannot learn non-linearly separable functions such as XOR.

- The learning rule updates weights and bias based on misclassified examples using a learning rate (eta).

Multilayer perceptron

A Multilayer Perceptron (MLP) is a feedforward artificial neural network architecture containing an input layer, one or more hidden layers of non-linear computational units, and an output layer. Connections flow strictly forward from input to output without directed cycles or feedback loops.

The Universal Approximation Theorem established by Cybenko and Hornik proves that a feedforward network with a single hidden layer containing a finite number of non-linear neurons can approximate any continuous function on compact subsets of Euclidean space to arbitrary precision.

Non-linear activation functions are essential because a multilayer network composed purely of linear activations collapses into a single-layer linear model regardless of depth. Intermediate hidden representations allow MLPs to learn complex non-linear decision boundaries and solve problems such as the XOR logic gate that single-layer perceptrons cannot learn.

activation functions

Non-linear activation functions are critical in neural networks because a multilayer architecture composed purely of linear activations collapses mathematically into a single-layer linear model regardless of depth. Non-linear activations introduce non-linear mapping capabilities necessary for complex function approximation.

Key activation functions include:

1. Sigmoid Activation: sigma(z) = 1 / (1 + exp(-z)). It maps inputs to the range (0, 1), functioning as a soft probability. Its derivative is sigma_prime(z) = sigma(z) * (1 - sigma(z)).
2. Hyperbolic Tangent (Tanh) Activation: tanh(z) = (exp(z) - exp(-z)) / (exp(z) + exp(-z)). It maps inputs to the range (-1, 1) and is zero-centered, which prevents directional bias in weight updates. Its derivative is tanh_prime(z) = 1 - tanh(z)^2.
3. Rectified Linear Unit (ReLU): f(z) = max(0, z). Returns zero for negative inputs and identity for positive inputs. ReLU eliminates negative saturation, accelerates gradient calculation, and enables deep architecture training.
4. Softmax Activation: For multi-class classification with K classes, Softmax(z_k) = exp(z_k) / Sum from j=1 to K of exp(z_j), producing a normalized posterior probability distribution.

network training

Training a neural network involves finding parameter weights W and biases b that minimize an empirical loss function J(W, b) over N training instances.

Primary loss functions include:

1. Mean Squared Error (MSE): Used for regression problems: J = (1 / (2*N)) * Sum from i=1 to N of ||y_i - y_hat_i||^2.
2. Binary Cross-Entropy: Used for binary classification: J = - (1 / N) * Sum from i=1 to N [y_i * ln(y_hat_i) + (1 - y_i) * ln(1 - y_hat_i)].
3. Categorical Cross-Entropy: Used for multi-class classification: J = - (1 / N) * Sum from i=1 to N of Sum from k=1 to K of [y_ik * ln(y_hat_ik)].

Network parameters are iteratively updated by descending the loss surface in the direction opposite to the loss gradient. Advanced optimizers incorporate momentum and adaptive learning rates to stabilize and accelerate convergence.

gradient descent optimization

Gradient descent optimization updates neural network parameter vectors theta = {W, b} iteratively in the direction of steepest descent across the empirical loss landscape: theta := theta - eta * grad(J(theta)), where eta > 0 represents the learning rate hyperparameter.

When evaluating loss functions, selecting an appropriate learning rate is crucial. Excessively large learning rates produce oscillations and divergence, while excessively small values lead to slow convergence.

Optimization variants balance computational speed and trajectory stability:

1. Batch Gradient Descent: Evaluates the entire dataset of N instances to compute the exact gradient before each weight update.
2. Momentum Optimization: Accumulates velocity vectors from past updates to dampen high-frequency oscillations across steep ravines.
3. Adaptive Optimizers: Methods such as RMSprop and Adam maintain running estimates of gradient moments to dynamically adjust individual parameter step sizes.

stochastic gradient descent

Stochastic Gradient Descent (SGD) updates parameters sequentially after computing the loss and gradient on a single randomly selected training instance (x_i, y_i): theta := theta - eta * grad_i(J(theta)).

SGD introduces high variance and stochastic noise into parameter trajectories, which helps the optimizer escape shallow local minima and saddle points. However, this noise also causes ongoing oscillation around optimal minima.

Mini-Batch Gradient Descent addresses these limitations by computing gradients across small subsets of size B (typically 32 to 256 samples). It combines the computational stability of batch methods with hardware vectorization benefits on GPUs, making it the standard training method for modern deep learning.

error backpropagation

The error backpropagation algorithm applies the multivariate chain rule of calculus to compute exact partial derivatives of the loss function J with respect to all network weights and biases.

The procedure consists of two sequential phases:

1. Forward Propagation: Input features x are propagated layer by layer to compute linear activations z^(l) = W^(l) * a^(l-1) + b^(l) and post-activations a^(l) = f^(l)(z^(l)), ending with prediction y_hat = a^(L).
2. Backward Propagation: Error vectors delta^(l) = dJ / dz^(l) are computed recursively. For the output layer, delta^(L) = y_hat - y under cross-entropy loss. For hidden layers: delta^(l) = ((W^(l+1))^T * delta^(l+1)) elementwise_product f_prime^(l)(z^(l)).

Parameter gradients are computed directly from error vectors: dJ / dW^(l) = delta^(l) * (a^(l-1))^T and dJ / db^(l) = delta^(l). Backpropagation achieves linear time complexity O(|E|) with respect to the number of network connections by caching intermediate activations during the forward pass.

from shallow networks to deep networks

While shallow networks with a single hidden layer possess theoretical universal approximation capabilities, representing complex functions with shallow topologies can require an exponentially large number of hidden neurons.

Deep neural networks overcome this limitation by organizing representations into hierarchical levels of abstraction:

1. Low-level layers capture primitive features such as localized edges and color boundaries.
2. Mid-level layers combine low-level primitives into textures, motifs, and part representations.
3. High-level layers compose complex semantic objects and global structures.

This hierarchical structure achieves superior parameter efficiency and generalization across vision, speech, and natural language domains. However, deeper networks require specialized architectural choices to address gradient instability during training.

Unit saturation (aka the vanishing gradient problem)

Unit saturation occurs in deep neural architectures when non-linear activation functions such as sigmoid or tanh enter regions where their derivative approaches zero.

During backpropagation across L layers, the error gradient propagated to the earliest layers involves a product of L derivative factors: delta^(1) is proportional to the Product from l=1 to L of f_prime^(l)(z^(l)).

Because the maximum derivative of the sigmoid function is 0.25 (and at most 1.0 for tanh), this cumulative product shrinks exponentially toward zero as network depth increases. As a result, early layers receive near-zero weight updates and fail to train effectively, a phenomenon known as the vanishing gradient problem. Conversely, large initial weights can cause gradients to grow exponentially, resulting in exploding gradients and numerical instability.

ReLU, hyperparameter tuning

The Rectified Linear Unit (ReLU) activation function, defined as f(z) = max(0, z), provides a practical solution to unit saturation. Because its derivative is a constant f_prime(z) = 1 for all positive inputs, backpropagated gradients flow through active units without exponential decay regardless of network depth. Variants like Leaky ReLU use a small slope for negative inputs to prevent dead neurons.

Hyperparameter tuning is essential for optimizing deep neural network performance. Key hyperparameters include learning rate eta, mini-batch size, network depth, hidden unit counts, and weight decay coefficients.

Standard search strategies include:

1. Grid Search: Evaluates exhaustive combinations across predefined discrete hyperparameter sets.
2. Random Search: Samples hyperparameter configurations randomly, often outperforming grid search in high-dimensional spaces.
3. Bayesian Optimization: Builds probabilistic models of objective performance to select new candidate configurations efficiently.

batch normalization, regularization, dropout.

Modern deep learning architectures utilize complementary techniques to stabilize optimization and prevent overfitting:

1. L2 Regularization (Weight Decay): Adds a penalty proportional to the sum of squared weights: J_reg = J + (lambda / (2*N)) * Sum of ||W||^2, which penalizes large parameter magnitudes to produce smoother decision boundaries.
2. Dropout: A stochastic regularization technique that randomly deactivates hidden units with probability p (typically 0.5) during each training forward pass. This prevents co-adaptation of features by requiring neurons to learn robust individual representations. At inference time, all units remain active with weights scaled by (1 - p).
3. Batch Normalization: Normalizes the pre-activations of each mini-batch to have zero mean and unit variance: z_norm = (z - mu_batch) / sqrt(sigma_batch^2 + epsilon). Learnable scale parameter gamma and shift parameter beta then restore model capacity: y_BN = gamma * z_norm + beta. Batch normalization reduces internal covariate shift, accelerates convergence, and acts as an implicit regularizer.

Source and licence

This PDF is adapted from **Artificial Intelligence and Machine Learning (Neural Networks and Deep Learning)** (https://nptel.ac.in/courses/106105077), accessed .

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).